



XOR a introwertyczność – rozwiązanie

Mamy dany ciąg liczb $(a_1, \dots, a_n)$ i chcemy wybrać z niego niektóre liczby tak, że każde dwie z nich mają wartość XOR większą lub równą danemu x . Liczba x (jak każda liczba) wygląda w zapisie binarnym jako tak:

$$x = \underbrace{00\dots 0}_{\text{początkowe}} \ 1 \ \underbrace{cc\dots c}_{\text{końcowe}}$$

Pogrubiona jedyńska to pierwsza jedyńska w zapisie x , miejsce (bit) na którym występuje, będziemy nazywać *kluczowym*. Bity wcześniejsze (x ma na nich zera) to będą bity *początkowe*, a późniejsze – *końcowe*.

Jeśli dwie liczby a_i oraz a_j różnią się gdziekolwiek na bitach początkowych, to $a_i \oplus a_j$ ma jedynekę na jednym z takich bitów, więc na pewno jest większe od x . Pogrupujmy zatem wszystkie dane liczby w *klasy* tak, że do jednej klasy trafiają wszystkie liczby o tych samych bitach początkowych. Do naszego podciągu możemy z każdej klasy wybierać liczby niezależnie, jako że między klasami XOR będzie odpowiedni. Wystarczy więc policzyć liczbę właściwych wyborów w każdej klasie osobno, a potem je pomnożyć przez siebie.

Skoncentrujmy się teraz na jednej konkretnej klasie, oznaczmy ją przez B , i podzielmy z kolei na B_0 i B_1 – liczby, które na kluczowym bicie mają odpowiednio 0 i 1. Do naszego podzbioru nie możemy wziąć dwóch liczb z B_0 , bo ich XOR ma 0 na kluczowym bicie, a więc jest mniejszy od x . Podobnie nie możemy wziąć dwóch liczb z B_1 . Zatem z klasy B możemy nie wziąć żadnej liczby (na jeden sposób), wziąć dokładnie jedną liczbę (na B sposobów), albo dokładnie dwie liczby, i wtedy to musi być para (u, v) taka, że $u \in B_0$, $v \in B_1$ i $u \oplus v \geq x$. Zostało nam zatem do obsłużenia tylko pytanie: dla danego $u \in B_0$, ile jest liczb v w B_1 takich, że $u \oplus v \geq x$? Jeśli umiemy na nie odpowiedzieć szybko, to możemy się przeiterować po wszystkich możliwych $u \in B_0$ i będziemy mieli kompletną odpowiedź dla całej klasy B , a więc dla wszystkich klas.

Skoro liczba $u \oplus v$ ma być większa lub równa x , to albo jest równa x (a wtedy wiemy, ile dokładnie musi być równe v – konkretnie $u \oplus x$), albo jest zgodna z x na pierwszych kilku bitach, a na którymś z końcowych bitów (powiedzmy o numerze j), musi zachodzić $x[j] = 0$, a $u \oplus v[j] = 1$. Dalsze bity mogą już być dowolne.

Przeiterujmy się zatem po wszystkich możliwych j , dla których $x[j] = 0$ (jest ich co najwyżej 63). Nasz warunek jest równoważny temu, że liczba v jest równa $u \oplus x$ na wszystkich bitach do $j - 1$ włącznie, zaś $u[j] \neq v[j]$. To jest po prostu pytanie, ile jest w zbiorze B_1 takich liczb v , które mają dane wartości na pierwszych j bitach. Na takie pytanie łatwo odpowiedzieć – liczby, które mają dane wartości na pierwszych j bitach, to po prostu liczby z ustalonego przedziału, można ich ilość znaleźć prostym wyszukiwaniem binarnym, jeśli posortujemy sobie wcześniej B_1 rosnąco. (Można też skonstruować tzw. *drzewo trie*, ale nie jest to konieczne.)

Podsumowując, dla każdego $u \in B_0$ i być może dla każdego bitu $j \leq 63$ musimy wykonać wyszukiwanie binarne. Całkowita złożoność nie przekroczy zatem $\mathcal{O}(\text{bity} \cdot n \log n)$ (a jeśli użyjemy drzewa trie, to znika logarytm).